Assessment 05 - Digital signatures

Instructions
The assessment must be developed individually and submitted via Blackboard by **18/11/2026**. The submission should be a *zip* file containing all developed code and a short PDF report describing the code and discussing the results.

This assessment takes into account the use of Python and the Cryptography library. Installation instructions can be found here. However, you can choose a language and library of your preference.

Activities

Q1: Message authentication codes (e.g., HMAC) and digital signatures both protect the integrity and the authenticity of a message. However, they do not offer the same guarantees.

Q1.a: Write a program that, for the same message, (i) generates and verifies an HMAC-SHA256 tag using a shared key `K`, and (ii) generates and verifies a digital signature using an RSA key pair (e.g., RSA-PSS with SHA-256). Compare both approaches in terms of key management, tag/signature size, and execution time (measure signing and verification over a large number of executions).
Q1.b: Consider a dispute in which Alice claims that Bob sent her a given message, and Bob denies it. Explain why HMAC cannot resolve this dispute (i.e., why it does not protect Bob from Alice), and why a digital signature can. Relate your answer to the *non-repudiation* property and to the requirement that *the signature must use data only known to the signer*.
Q1.c: Bob encrypts a message with Alice's public key (`PU_Alice`) and sends it to her. Does this scheme authenticate the source? Show, with a short code example, how an attacker could send Alice a valid ciphertext pretending to be Bob.

Q2: Signing with RSA directly over the message (i.e., without hashing and without padding, the so-called *textbook RSA* signature, `s = m^d mod n`) violates one of the requirements of digital signature schemes: *it must be computationally infeasible to forge a digital signature*.

Q2.a: Implement textbook RSA signing and verification using Python integers (`pow(m, d, n)` and `pow(s, e, n)`), with keys of at least 2048 bits generated by the Cryptography library (use `private_numbers()` and `public_numbers()` to obtain `n`, `e`, and `d`). For simplicity, represent each message as an integer `m` with `0 < m < n`.
Q2.b: Given two valid signatures `s1` and `s2` on messages `m1` and `m2`, show how an attacker who does not know `d` can produce a valid signature on a third message. Demonstrate the attack with your implementation. Which type of forgery from the slides (*new message for an existing signature* or *fraudulent signature for a given message*) does it correspond to?
Q2.c: Explain why the generic approach `E(PR_a, H(M))`, combined with a padding scheme such as PSS, prevents this attack.

Q3: The DSA signature function receives as input a random number `k` that must be generated for each particular signature. The following two signatures were produced by the same signer, using the global public-key components and the public key below. Note that both signatures share the same value of `r`.

```
Global public-key components (PU_G):
p =
0x90d5e9facf54c404d5056469a3e7c7e425f93ed258f04a4b6217eabd9aa3acaacadb4059ef50fdf63ad42f8fad2857ee8f3f537c0628
608f630342071b30724a94f88feb5faafd30a72221984f057749c0e8782e4492a6293c9e52e566c5222c5aef569098d1f501a2375ac07e
e32725c4fb15f77c33a58f17fe4bd96b49a8bb
q = 0x8b6bd7869151052c5de1d9e9771ff8a07b6d674b
g =
0x5b8b967cb90f9b488755a40b9831a981ee9a5486a3afd96ad7f8b86e35b8116c5dda04d4bfc2a7044fcea37f723e6406730c649e753e
23c257e112bad0a9275317b433ec4de2cdc28c2d2c21c90008734eabecca0d60d490b02ab458f25d4f83aab06fcacba327b158aa0fe437
54641dbfcf12b01952d30b1770ba4e5b1c4a33

Signer's public key:
y =
0x466b1cbd3c016447d484c171e67d171a2ddb296b6546f1e0c0ebc47fc3d02217dc25b3dc34d3bf6ea41ffd25d04bc47f6880a07e663a
b0eb8924193ef2f90b46d76ed3faeee44684f13a676afdacd0f82162f81ca210b3bc1914b8ef78ab531c5b7d5a44b984b6f23d9487290a
6515116105617f0b5661c0d14f16da0d77c31f

Message 1: "TRANSFER 500 EUR TO ACCOUNT PT50-0001"
Signature 1: (r, s1)
r  = 0x4b14d71b39ae81b4110c93f0d34930e041ec5c94
```

```
s1 = 0x58f765a04b4aaa239c1e34ff7c13066dea00ea8d

Message 2: "TRANSFER 20 EUR TO ACCOUNT PT50-0002"
Signature 2: (r, s2)
r  = 0x4b14d71b39ae81b4110c93f0d34930e041ec5c94
s2 = 0x81cc418f55b266a5cb4965c0d9321e1831ac03da
```

Consider H(M) = int(SHA-1(M)) mod q, where M is encoded in UTF-8 and the SHA-1 digest is interpreted as a big-endian integer. All values are represented in hexadecimal.

Q3.a: Implement the DSA verification algorithm described in the slides and confirm that both signatures are valid.
Q3.b: What does a shared value of `r` reveal about how the signatures were produced? Based on the signing equation `s = [k^-1 (H(M) + xr)] mod q`, derive the expressions that allow recovering the per-message secret number `k` and then the private key `x`.
Q3.c: Write the program `dsa_attack.py` that recovers `x`, checks it against the public key (`y = g^x mod p`), and uses it to produce a valid signature for the message `"TRANSFER 99999 EUR TO ACCOUNT PT50-6666"`. Verify the forged signature with your implementation from *Q3.a*.
